# LAPORAN
# WORKSHOP SISTEM INFORMASI WEB SERVER
# (TIF320803)
# MINGGU 8 ACARA 18

## SEMESTER 3



**Tugas :**
**JSON encoding dan HTTP status code**

**Disusun Oleh:**
**Uaryl Engno Laurenca ( E41250317)**

**PROGRAM STUDI DILUAR KAMPUS UTAMA (PSDKU)**
**TEKNIK INFORMATIKA KAB. NGANJUK**
**JURUSAN TEKNOLOGI INFORMASI**
**POLITEKNIK NEGERI JEMBER**
**TAHUN 2026**

## A. PENDAHULUAN

Pada Acara 17 telah dibangun API CRUD dengan PHP Native berpola MVC. Pada praktikum itu, setiap Controller masih menyusun response sendiri-sendiri dengan memanggil `http_response_code()` dan `echo json_encode()` secara langsung. Cara ini membuat kode berulang dan format response mudah berbeda-beda antar endpoint, sehingga sisi client harus menangani banyak variasi struktur data.

Acara 18 membahas dua hal yang saling terkait. Pertama, JSON encoding, yaitu cara mengubah data PHP menjadi teks JSON dengan opsi yang tepat. Kedua, HTTP status code, yaitu kode numerik yang memberi tahu client hasil pemrosesan request. Keduanya diterapkan dengan membuat satu class `Response` pada lapisan Core, lalu merefaktor Controller dan Router agar memakainya. Dengan begitu seluruh response API memiliki format seragam (response envelope), sedangkan lapisan Model tidak perlu diubah.

## B. TUJUAN PRAKTIKUM

1. Memahami fungsi json_encode() dan json_decode() beserta flag JSON_PRETTY_PRINT, JSON_UNESCAPED_SLASHES, dan JSON_UNESCAPED_UNICODE.
2. Memahami lima kategori HTTP status code (1xx sampai 5xx) dan kode yang sering dipakai pada REST API.
3. Membuat class Response sebagai pembentuk response JSON yang seragam.
4. Merefaktor Controller dan Router agar memakai Response::json().
5. Menerapkan validasi dengan kode 422 (Unprocessable Entity) dan membedakannya dari 400.
6. Menambahkan field errors untuk detail kesalahan per field, serta mengatur pretty print hanya pada mode development.
7. Menguji ulang seluruh endpoint dan skenario gagal dengan Postman.

## C. DASAR TEORI

### JSON Encoding dan Decoding pada PHP

json_encode() mengubah array atau objek PHP menjadi teks JSON, sedangkan json_decode() melakukan proses sebaliknya. Flag tambahan yang dipakai:

- JSON_PRETTY_PRINT : menata hasil JSON dengan indentasi agar mudah dibaca
- JSON_UNESCAPED_SLASHES : karakter garis miring tidak di-escape
- JSON_UNESCAPED_UNICODE : karakter non-ASCII ditampilkan apa adanya, bukan kode unicode

### HTTP Status Code

HTTP Status Code adalah kode numerik dari server tentang hasil pemrosesan request. Lima kategorinya:

- 1xx Informational
- 2xx Success

- 3xx Redirection
- 4xx Client Error
- 5xx Server Error
- Kode yang sering dipakai pada REST API:
- 200 OK : GET/PUT/DELETE berhasil
- 201 Created : data baru berhasil dibuat lewat POST
- 204 No Content : berhasil tanpa isi
- 400 Bad Request : format data pada request tidak valid
- 401 Unauthorized : tidak menyertakan autentikasi yang sah
- 403 Forbidden : tidak punya hak akses
- 404 Not Found : data atau endpoint tidak ditemukan
- 405 Method Not Allowed : method tidak diperbolehkan
- 409 Conflict : terjadi konflik data
- 422 Unprocessable Entity : data valid secara format tetapi gagal validasi bisnis
- 500 Internal Server Error : kegagalan tak terduga di server

Di PHP Native, kode dikirim dengan http_response_code(200) atau http_response_code(404). Penerapan yang konsisten mempermudah debugging, meningkatkan interoperabilitas, membantu client mengambil keputusan, dan mengikuti standar HTTP**.**

**Standardized Response Envelope**
Response envelope adalah format response yang konsisten, umumnya berisi indikator keberhasilan (success), data hasil (data), pesan (message), dan metadata seperti timestamp. Response error sebaiknya juga memuat rincian tiap field yang gagal validasi. Konsistensi ini memudahkan client memproses response tanpa menangani banyak variasi struktur.

**RFC 9457 (Problem Details for HTTP APIs)**
RFC 9457 (Maret 2023) adalah standar IETF untuk format JSON response error, dengan lima bidang utama: type, title, status, detail, dan instance, memakai tipe konten application/problem+json. Praktikum ini memakai envelope yang lebih sederhana, tetapi standar ini perlu dipahami untuk pengembangan API tingkat lanjut atau API publik.

## D. HASIL DAN PEMBAHASAN

**Alat dan Bahan**
XAMPP (Apache, PHP, MySQL), Visual Studio Code, Git dan GitHub, web browser, dan Postman.

**Gambaran Perubahan**
Acara ini menambahkan satu class baru di lapisan Core dan merefaktor beberapa file. Lapisan Model dan database tidak diubah.
- app/core/Response.php : baru, pembentuk response JSON
- app/config/app.php : baru, konstanta APP_ENV dan zona waktu
- app/controllers/MahasiswaController.php : memakai Response, validasi 400 dan 422
- app/controllers/BukuController.php : memakai Response, validasi 400 dan 422
- app/core/BaseController.php : readJson() dan requireId() memakai Response

- app/core/Router.php : blok 404 dan 405 memakai Response
- app/config/Database.php : gagal koneksi memakai Response (500)
- public/index.php : memuat app.php, error tak terduga dibalas 500 lewat Response

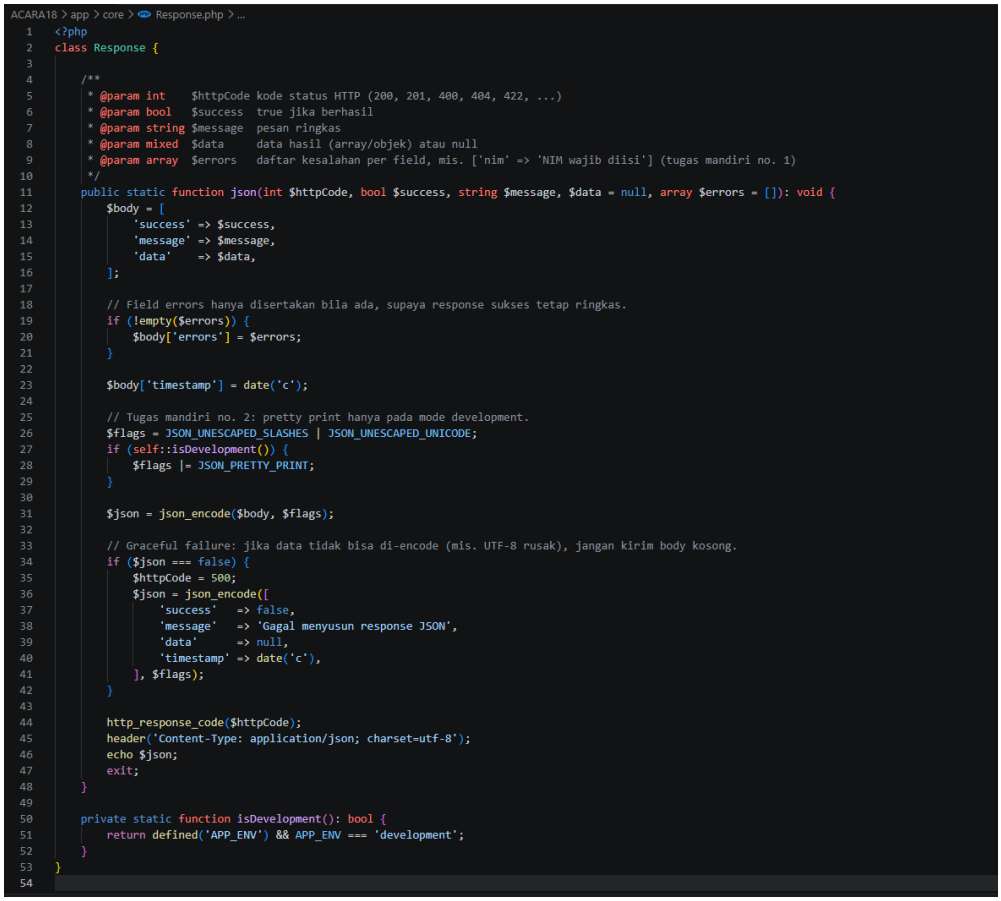
**Langkah 1: Membuat Class Response**

File app/core/Response.php berisi method statis Response::json($httpCode, $success, $message, $data = null, $errors = []). Method ini mengirim status code dengan http_response_code(), mengirim header Content-Type: application/json, lalu menampilkan response dengan json_encode() dan diakhiri exit. Class ini otomatis ditemukan autoloader karena folder core sudah terdaftar sejak Acara 17.

Format response (envelope):

success (true/false), message (pesan), data (hasil atau null), errors (hanya jika ada), dan timestamp (waktu response).

Sebagai pengaman, jika data gagal di-encode (misalnya karakter UTF-8 rusak), class mengirim response 500 yang sah, bukan body kosong.




```php
<?php
class Response {

    /**
     * @param int    $httpCode kode status HTTP (200, 201, 400, 404, 422, ...)
     * @param bool   $success  true jika berhasil
     * @param string $message  pesan ringkas
     * @param mixed  $data     data hasil (array/objek) atau null
     * @param array  $errors   daftar kesalahan per field, mis. ['nim' => 'NIM wajib diisi'] (tugas mandiri no. 1)
     */
    public static function json(int $httpCode, bool $success, string $message, $data = null, array $errors = []): void {
        $body = [
            'success' => $success,
            'message' => $message,
            'data'    => $data,
        ];

        // Field errors hanya disertakan bila ada, supaya response sukses tetap ringkas.
        if (!empty($errors)) {
            $body['errors'] = $errors;
        }

        $body['timestamp'] = date('c');

        // Tugas mandiri no. 2: pretty print hanya pada mode development.
        $flags = JSON_UNESCAPED_SLASHES | JSON_UNESCAPED_UNICODE;
        if (self::isDevelopment()) {
            $flags |= JSON_PRETTY_PRINT;
        }

        $json = json_encode($body, $flags);

        // Graceful failure: jika data tidak bisa di-encode (mis. UTF-8 rusak), jangan kirim body kosong.
        if ($json === false) {
            $httpCode = 500;
            $json = json_encode([
                'success'   => false,
                'message'   => 'Gagal menyusun response JSON',
                'data'      => null,
                'timestamp' => date('c'),
            ], $flags);
        }

        http_response_code($httpCode);
        header('Content-Type: application/json; charset=utf-8');
        echo $json;
        exit;
    }

    private static function isDevelopment(): bool {
        return defined('APP_ENV') && APP_ENV === 'development';
    }
}
```

*SCREENSHOT 2: kode Response.php*

**Langkah 2: Refactor Controller**

Seluruh pemanggilan http_response_code() dan echo json_encode() di Controller diganti dengan Response::json(). Kode Controller menjadi jauh lebih ringkas karena detail pembentukan JSON kini menjadi tanggung jawab Core. Contohnya, pencarian data yang tidak ditemukan cukup ditulis Response::json(404, false, 'Data tidak ditemukan'), dan penambahan data berhasil ditulis Response::json(201, true, 'Data berhasil ditambahkan', ['id' => $id]).




*SCREENSHOT 3: kode MahasiswaController.php setelah refactor*

**Langkah 3: Validasi dengan Kode 422**

Validasi dibuat dua tahap agar kode status tepat:

1. Tahap 1 (400 Bad Request): field nim, nama, atau prodi kosong atau tidak dikirim.
2. Tahap 2 (422 Unprocessable Entity): JSON lengkap tetapi isinya melanggar aturan, yaitu NIM tidak tepat 9 karakter, nama lebih dari 100 karakter, atau prodi lebih dari 50 karakter.

Aturan NIM mengikuti modul (strlen($data['nim']) !== 9). Pemeriksaan sengaja hanya pada panjang, tidak membatasi angka saja, karena NIM dapat memuat huruf, misalnya E41250317. Pada Controller Buku, tahun yang bukan angka 4 digit valid juga dibalas 422.

```php
    /**
     * Validasi dua tahap:
     *  1) 400 Bad Request          : field wajib kosong / tidak dikirim
     *  2) 422 Unprocessable Entity : JSON lengkap, tetapi isinya gagal validasi (format NIM, panjang)
     * Daftar kesalahan per field dikirim lewat parameter $errors pada Response::json().
     */
    private function validated(array $in): array {
        $data = [];
        foreach (['nim', 'nama', 'prodi'] as $field) {
            $data[$field] = isset($in[$field]) && is_scalar($in[$field]) ? trim((string) $in[$field]) : '';
        }

        // Tahap 1: field wajib
        $missing = [];
        foreach ($data as $field => $value) {
            if ($value === '') {
                $missing[$field] = "Field $field wajib diisi";
            }
        }
        if (!empty($missing)) {
            Response::json(400, false, 'Field nim, nama, dan prodi wajib diisi', null, $missing);
        }

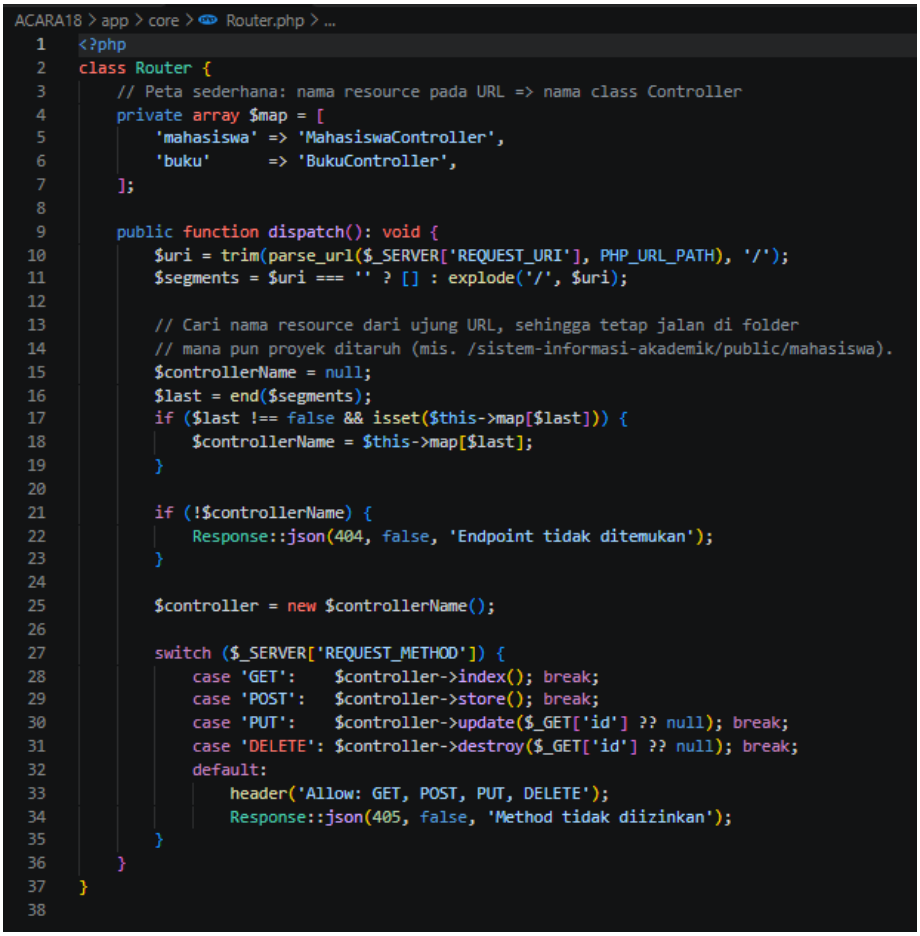
        // Tahap 2: format / aturan data
        $errors = [];
        // Modul: NIM harus tepat 9 karakter (strlen !== 9). Sengaja tidak dibatasi angka saja
        // karena NIM Polije dapat memuat huruf, mis. E41250317.
        if (strlen($data['nim']) !== 9) {
            $errors['nim'] = 'Format NIM tidak valid, harus 9 digit';
        }
        if (mb_strlen($data['nama']) > 100) {
            $errors['nama'] = 'Nama maksimal 100 karakter';
        }
        if (mb_strlen($data['prodi']) > 50) {
            $errors['prodi'] = 'Prodi maksimal 50 karakter';
        }
        if (!empty($errors)) {
            Response::json(422, false, 'Data tidak lolos validasi', null, $errors);
        }

        return $data;
    }
}
```

*SCREENSHOT 4: bagian validasi pada MahasiswaController.php*

**Langkah 4: Merapikan Response pada Router**

Blok 404 (endpoint tidak ditemukan) dan 405 (method tidak diizinkan) pada Router diganti dengan Response::json(). Pada 405, header Allow: GET, POST, PUT, DELETE tetap dikirim. Hasilnya, response dari Router dan Controller memiliki format yang sama persis.



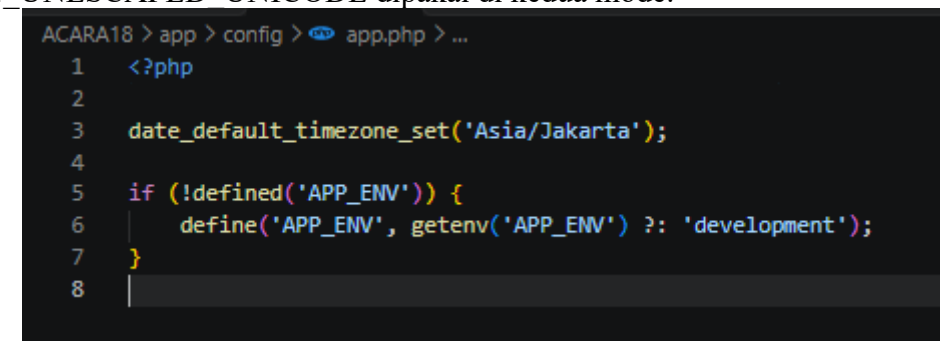

*SCREENSHOT 5: kode Router.php*

**Tugas Mandiri 1: Field errors**

Parameter ke-5 Response::json() berupa array kesalahan per field, misalnya ['nim' => 'Format NIM tidak valid, harus 9 digit']. Field errors hanya dimasukkan ke response jika array tidak kosong, sehingga response sukses tetap ringkas. Dengan field ini, client bisa menampilkan pesan kesalahan tepat di kolom formulir yang bermasalah.

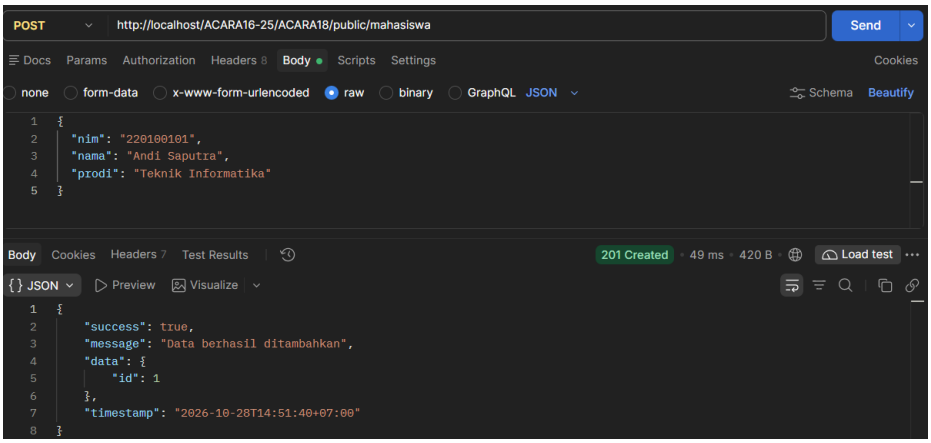
**Tugas Mandiri 2: Pretty Print Hanya pada Mode Development**

Dibuat file konfigurasi baru app/config/app.php yang mendefinisikan konstanta APP_ENV (nilai ‘development’ atau ‘production’) dan zona waktu Asia/Jakarta. Class Response hanya menambahkan flag JSON_PRETTY_PRINT jika APP_ENV bernilai ‘development’. Pada mode production, JSON keluar dalam satu baris sehingga lebih hemat bandwidth. Flag JSON_UNESCAPED_SLASHES dan JSON_UNESCAPED_UNICODE dipakai di kedua mode.



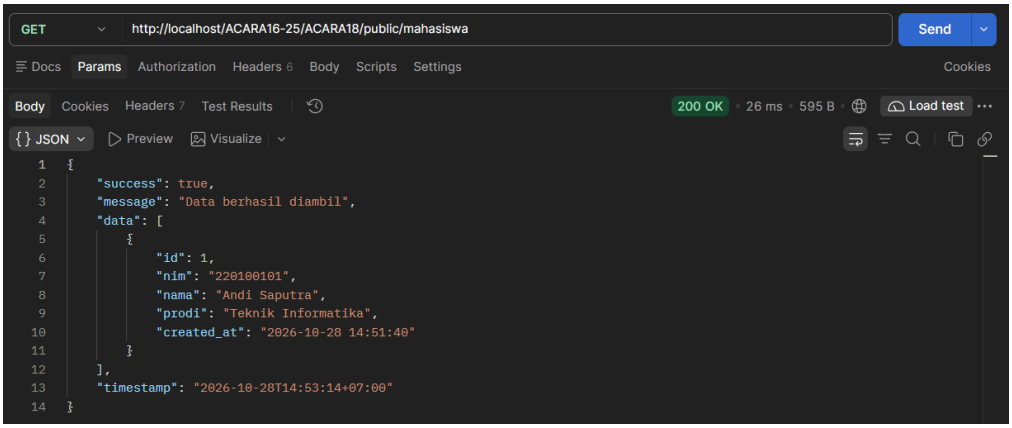

*SCREENSHOT 6: kode app.php*

**Langkah 5: Pengujian Ulang dengan Postman**
Base URL: http://localhost/ACARA16-25/ACARA18/
**A. Skenario berhasil (data uji: NIM 220100101)**
(1) POST /mahasiswa dengan {“nim”:”220100101",”nama”:”Andi Saputra”,”prodi”:”Teknik Informatika”}
Hasil: status 201 Created, success true, message “Data berhasil ditambahkan”, data berisi id, dan timestamp.



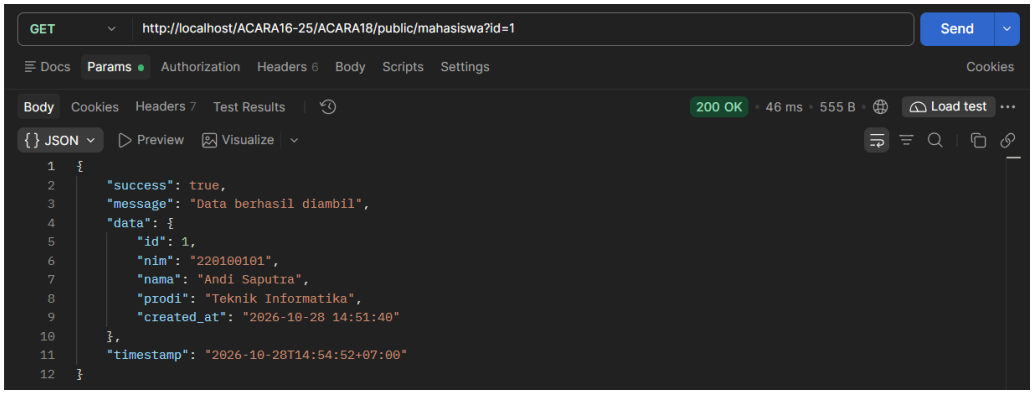
*[SCREENSHOT 7: Postman POST, body dan response 201]*

(2) GET /mahasiswa
Hasil: status 200 OK, data berupa array seluruh mahasiswa.



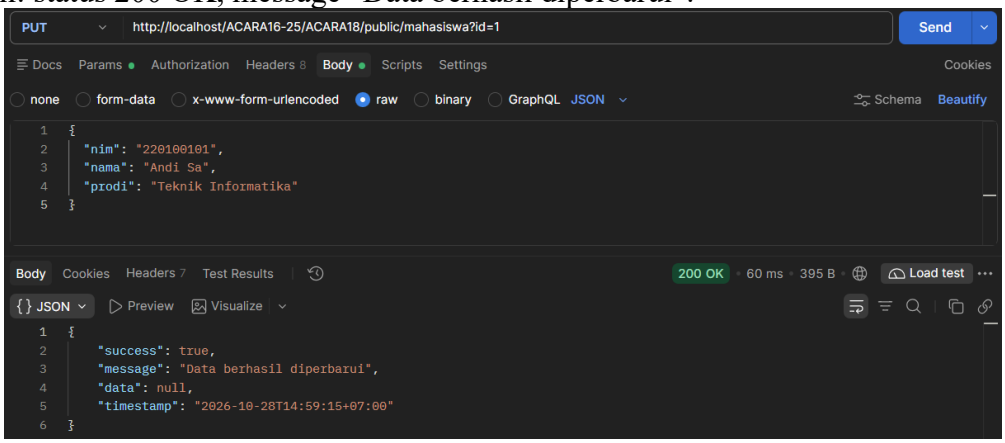
*[SCREENSHOT 8: Postman GET semua data, status 200]*

(3) GET /mahasiswa?id=1
Hasil: status 200 OK, data satu mahasiswa.



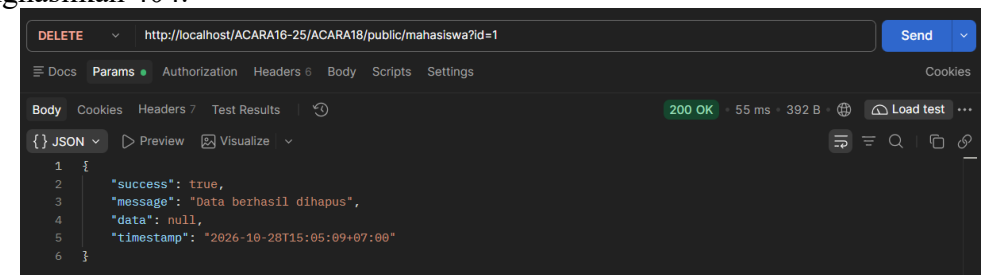

*[SCREENSHOT 9: Postman GET ?id=1, status 200]*

(4) PUT /mahasiswa?id=1 dengan data baru
Hasil: status 200 OK, message “Data berhasil diperbarui”.

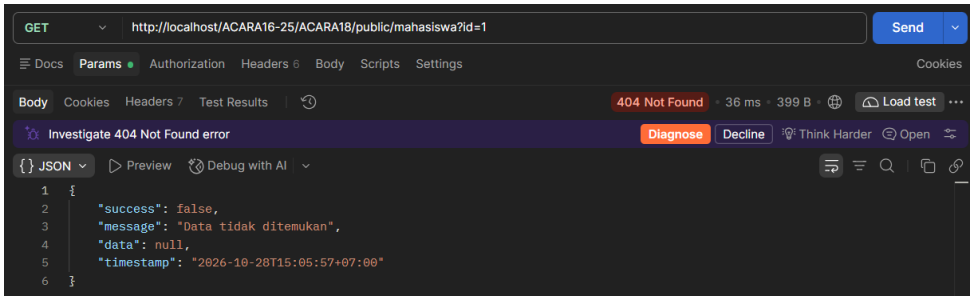



*[SCREENSHOT 10: Postman PUT, status 200]*

(5) DELETE /mahasiswa?id=1
Hasil: status 200 OK, message “Data berhasil dihapus”. Setelah itu GET ?id=1 menghasilkan 404.



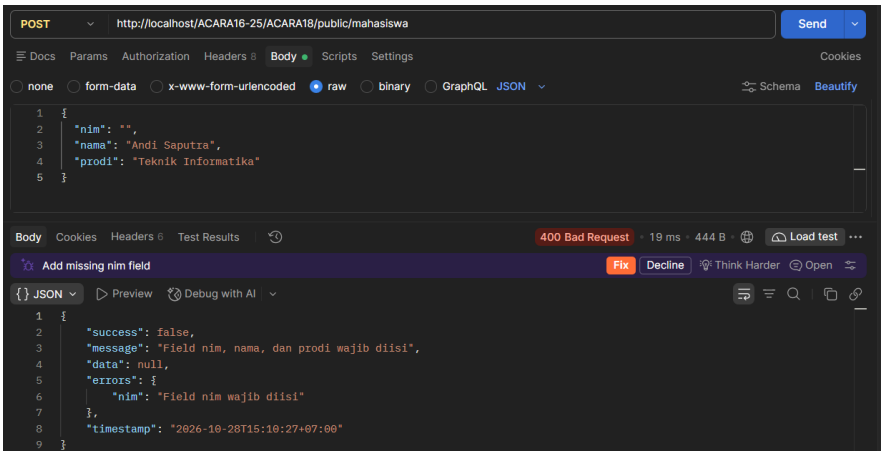

*[SCREENSHOT 11: Postman DELETE, status 200]*




*[SCREENSHOT 12: Postman GET ?id=1 setelah delete, status 404]*

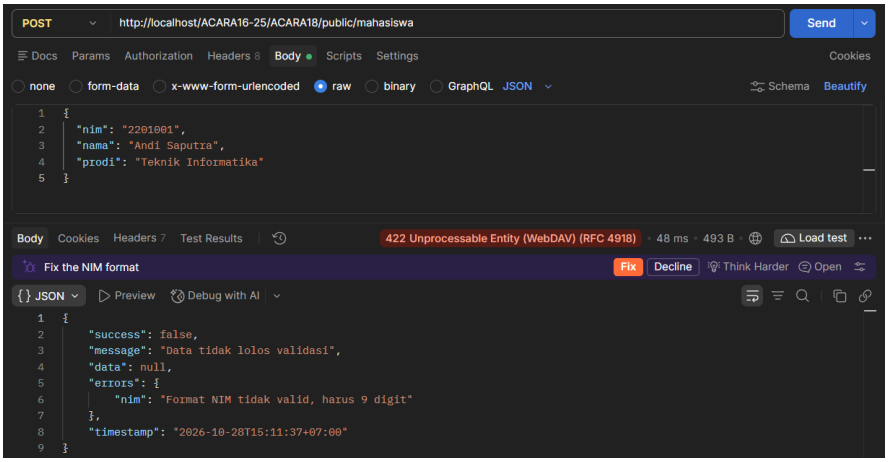
**B. Skenario gagal**

(1) Data kosong: POST dengan nim “” menghasilkan 400 Bad Request, message “Field nim, nama, dan prodi wajib diisi”, dan errors berisi {“nim”: “Field nim wajib diisi”}.



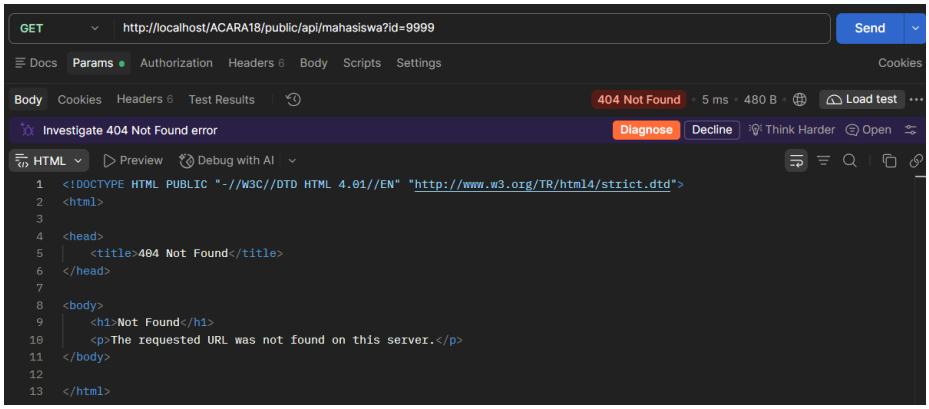

*[SCREENSHOT 13: Postman POST nim kosong, status 400, tampilkan field errors]*

(2) Format NIM salah: POST dengan nim “2201001” (7 karakter) menghasilkan 422 Unprocessable Entity, message “Data tidak lolos validasi”, dan errors berisi {“nim”: “Format NIM tidak valid, harus 9 digit”}.



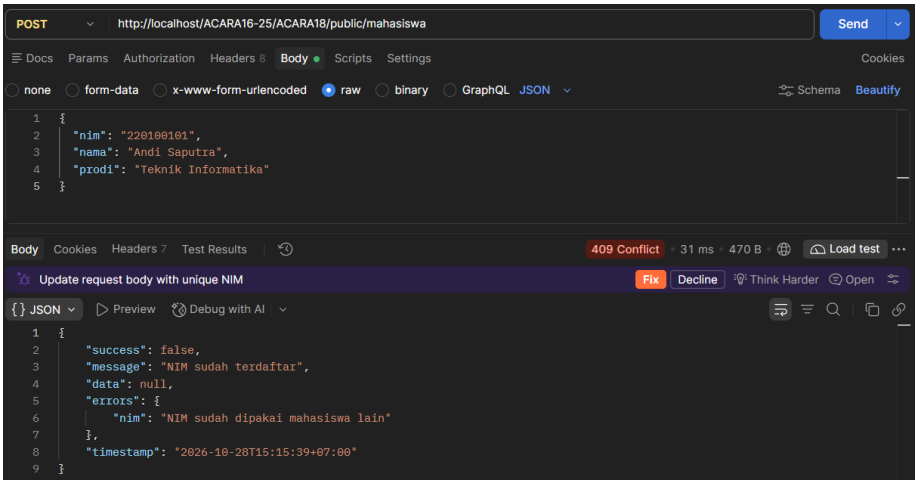

*[SCREENSHOT 14: Postman POST NIM 7 digit, status 422]*

(3) ID tidak ditemukan: GET ?id=9999 menghasilkan 404 Not Found, message “Data tidak ditemukan”.



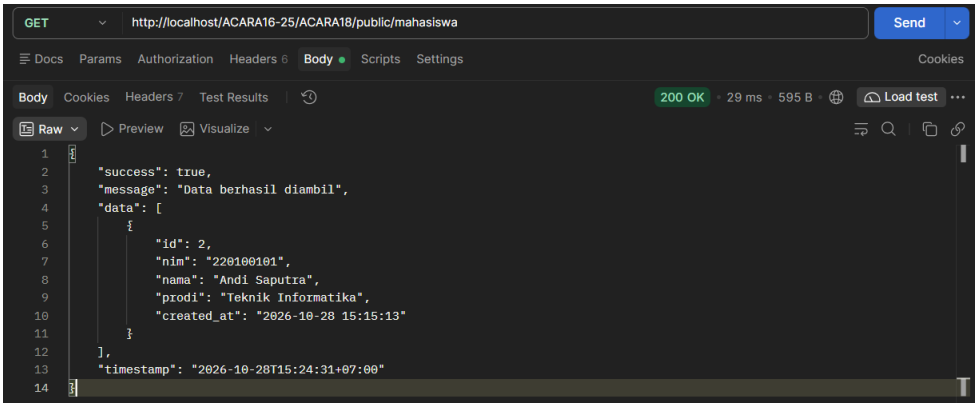

*[SCREENSHOT 15: Postman GET ?id=9999, status 404]*

(4) NIM duplikat: POST dengan NIM yang sudah ada menghasilkan 409 Conflict.

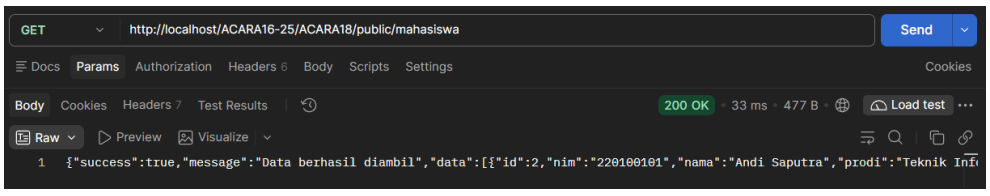



*[SCREENSHOT 16: Postman POST NIM duplikat, status 409]*

C. Pengujian mode development dan production
Dengan APP_ENV ‘development’, response tampil rapi berindentasi. Setelah diubah menjadi ‘production’, response yang sama tampil dalam satu baris.




*[SCREENSHOT 18: response mode development (rapi) di Postman]*




*[SCREENSHOT 19: response mode production (satu baris) di Postman]*

D. Collection Runner
Seluruh request tersimpan dalam satu Collection yang memiliki test otomatis untuk status code, header Content-Type, kelengkapan envelope (success, message, data, timestamp), dan keberadaan field errors pada response gagal.

**Pembahasan**

1. Satu pintu pembentukan response. Dengan Response::json(), format JSON, header, dan status code diatur di satu tempat. Jika format ingin diubah, cukup mengubah satu file, tidak perlu menyentuh setiap Controller. Ini penerapan prinsip DRY (Don't Repeat Yourself) dan pemisahan tanggung jawab.
2. Perbedaan 400 dan 422. Kode 400 dipakai ketika request itu sendiri bermasalah (data tidak ada atau bukan JSON valid), sedangkan 422 dipakai ketika request dapat dibaca tetapi isinya tidak memenuhi aturan (NIM bukan 9 karakter). Pembedaan ini membantu client mengetahui jenis kesalahan tanpa membaca isi pesan.
3. Kode 409 untuk NIM duplikat. Masalahnya bukan format data, tetapi konflik dengan data yang sudah ada. Karena itu 409 lebih tepat daripada 400 atau 422.
4. Field errors. Pesan per field memungkinkan client menampilkan kesalahan tepat di kolom yang bermasalah, bukan hanya satu pesan umum.
5. Mode development dan production. Pretty print mempermudah pembacaan saat pengembangan, sedangkan JSON ringkas mengurangi ukuran response di lingkungan produksi. Perbedaan ini diatur lewat konfigurasi, bukan dengan mengubah kode.
6. Graceful failure. Detail error server (misalnya koneksi database gagal) hanya dicatat di log, sedangkan client menerima pesan umum dengan status 500. Jika json_encode() gagal, Response tetap mengirim JSON yang sah.
7. Hubungan dengan RFC 9457. Format envelope pada praktikum ini lebih sederhana dibanding RFC 9457, tetapi memiliki ide yang sama: error yang konsisten dan dapat diproses mesin. Untuk API publik berskala besar, format RFC 9457 dapat diadopsi.

## E. KESIMPULAN

Class `Response` berhasil dibuat untuk menyeragamkan format response API, baik dari Router maupun Controller, dengan struktur envelope yang terdiri dari `success`, `message`, `data`, `errors`, dan `timestamp`. Penerapan HTTP status code disesuaikan dengan jenis respons, yaitu 200 dan 201 untuk keberhasilan, 400, 404, 405, 409, dan 422 untuk kesalahan client, serta 500 untuk kesalahan server. Validasi dilakukan dalam dua tahap, yaitu status 400 ketika data tidak tersedia dan 422 ketika data tidak memenuhi aturan validasi, sehingga client dapat membedakan jenis kesalahan yang terjadi. Tugas mandiri juga berhasil diselesaikan dengan menerapkan field `errors` untuk menampung pesan kesalahan berdasarkan masing-masing field serta mengaktifkan fitur pretty print hanya pada mode development melalui konstanta `APP_ENV`. Proses refactoring hanya dilakukan pada lapisan Core, Controller, dan konfigurasi tanpa mengubah lapisan Model. Hal ini menunjukkan bahwa pemisahan tanggung jawab dalam arsitektur MVC telah berjalan dengan baik sehingga kode program menjadi lebih terstruktur, konsisten, dan mudah dikembangkan.